RESEARCH-ARTICLE

Code comprehension problems as learning events

LEIGH ANN SUDOL-DELYSER, Carnegie Mellon University, Pittsburgh, PA, United States

MARK STEHLIK, Carnegie Mellon University, Pittsburgh, PA, United States

SHARON CARVER, Carnegie Mellon University, Pittsburgh, PA, United States

Published: 03 July 2012

ITiCSE '12: Annual Conference on Innovation and Technology in Computer Science Education
July 3 - 5, 2012
Haifa, Israel

Conference Sponsors:
SIGCSE

ITiCSE '12: Proceedings of the 17th ACM annual conference on Innovation and technology in computer science education (July 2012)
https://doi.org/10.1145/2325296.2325319
ISBN: 9781450312462

Code Comprehension Problems as Learning Events

Leigh Ann Sudol
Computer Science Department
Carnegie Mellon University
Pittsburgh, PA
lsudol@andrew.cmu.edu

Mark Stehlik
Computer Science Department
Carnegie Mellon University
Pittsburgh, PA
mjs@cs.cmu.edu

Sharon Carver
Psychology Department
Carnegie Mellon University
Pittsburgh, PA
sc0e@andrew.cmu.edu

ABSTRACT

Code comprehension problems have been shown to be effective assessment items in computer science education. In this paper we present qualitative and quantitative results of a study evaluating the effectiveness of code comprehension questions with feedback as learning events. Students taking an introductory programming course that satisfies a university requirement interacted with an online tutoring system using code comprehension problems about simple array algorithms as a part of a homework assignment. Students answered the problems in their own words first, before selecting a multiple choice option from the system.

Both the open-ended and multiple-choice responses were collected and analyzed. Results indicate that code comprehension questions with appropriate feedback can be learning events. The use of open-ended and multiple choice responses to the same question is also shown to be useful in refining distracter items for future assessment. Recommendations from this study can be applied not only to tutoring systems, but also to the type of interactions used in worked examples in class lecture and textbook production.

Categories and Subject Descriptors

I.2.6 [**Computing Methodologies**]: Learning;Concept Learning; K.3.2 [**Computers & Education**]: Computer and Information Sciences Education;Computer Science Education

General Terms

Human Factors

Keywords

Code Comprehension, Worked Examples, Tutoring System, Feedback

ITiCSE'12, July 3–5, 2012, Haifa, Israel.

1. INTRODUCTION

Code comprehension problems are often used for assessment in computer science education, especially at the introductory level [1]. These problems require the student to read a segment of code, and then answer a question about the code. There are many types of code comprehension problems, as described in 1.1, and each type has different assessment implications. Code comprehension questions are an often-employed method of assessment because performance on code comprehension questions often correlates with code production or code writing [2]. Code comprehension questions also allow for more detailed analysis of student ability than code production [3] because descriptions of code in natural language can expose mental models and misconceptions held by the student.

Although code comprehension problems are a popular assessment item, little work has been done to evaluate their viability as a learning task when accompanied by appropriate feedback. Textbooks and teachers often use worked examples during instruction [4], and many of the papers published about peer instruction [5] include code comprehension questions in the instruction.

The work presented in this paper explores code comprehension questions that are learning tasks as a part of an online computer-supported tutoring assignment. Both open ended responses and multiple choice selections are analyzed, and results indicate appropriate code comprehension questions with feedback can produce both increasingly accurate multiple choice answers and more sophisticated open ended responses over time.

1.1 Code Comprehension Problems

Code comprehension problems are presented in many different ways. The type of question asked about the code varies depending on the goals of the assessment [6]. Students may be asked to trace the code with a specified set of input values to determine output, reason about the code abstractly (without specified inputs) to make a generic statement about the output, produce invariants about the code's performance, or describe the code in their own words often classifying the algorithm in the process. Just as the questions can vary, the type of responses expected and the grading of those responses will vary as well [7]. Two of the most common types of responses are multiple choice and open ended.

Both multiple choice and open ended responses have benefits and weaknesses. Multiple choice responses are easy to grade, especially by automated systems, while open ended

responses are more difficult to score automatically. Multiple choice however constrains the possibilities offered the student and makes it more possible to guess the correct answer, while an open ended response will give more detail about a student's true mental model. Open ended responses also provide the opportunity to classify the student's level of reasoning when answering the problem against one of several appropriate taxonomies [8].

There is a difference between the ability to trace code and reason about code. Tracing can be seen as a precursor skill to reasoning about code [9]. However, tracing ability alone is not sufficient for skill at writing code, as students must also be able to explain the code and the abstractions contained within to develop code production skills [10].

The tutoring environment used for this study contained two types of code comprehension questions, tracing and algorithmic abstraction questions. In the tracing questions, students are presented with a code snippet and input values for any parameters or variables required. The students are then asked to calculate the resulting value contained in a variable after the code has executed. Algorithmic abstraction questions focus on primarily on two key student abilities. One type of algorithmic abstraction question asks the student to make a general observation about the behavior of the code (What does this code do? It finds the sum of all the numbers in the list). The second type of algorithmic abstraction question asks about the role of a particular variable within the algorithm. The three questions analyzed in Section 3 ask about the role of variables related to the accessing of individual elements within the arrays used in the questions.

1.2 Questions as Learning Events

There is a rich history in intelligent tutoring systems (ITS) of using questioning with feedback as learning events for students [11, 12]. Within computer science education, many of the tutors have focused on the code writing ability of students [13, 14, 15, 16]. Many experts believe that in addition to code production, code comprehension is an important skill for novices to learn [17, 18]. In addition to using comprehension skills to troubleshoot and debug their own code, comprehension skills are also important for parsing and integrating concepts demonstrated in worked examples. In addition, code comprehension problems themselves can be viewed as worked examples, and the questions asked about the code can draw a student's attention away from surface features of the problem, to the deeper algorithmic abstractions in the example. There is evidence for the need to explicitly draw students' attention to the algorithms, as students seem not to make the connection from simple tracing problems and translate the observed algorithm to code production [10].

The tutor discussed in this work uses code comprehension questions and is part of a larger study published in [19]. The goal of the larger study was to evaluate the impact on student learning of questions regarding algorithmic abstractions compared to tracing questions on the same code. We focus here specifically on open-ended student responses to a series of similar algorithmic abstraction questions in the tutor.[1] We describe the change that occurs through the sequence of questions, highlighting the improved performance and explanations students provide as they encounter learning events involving questions with feedback.

[1]Student responses to the tracing questions were mostly the singular value the function returned and therefore we deemed them uninteresting for this analysis.

2. METHODOLOGY

This paper includes data collected as a part of a larger study [19] focused on instructing students in algorithmic abstractions. Participants engaged with an online tutoring system comprised of 30 multiple choice questions as a part of a larger homework assignment in an introductory computer science course. As a part of the data collection, students were asked to submit open ended responses to the questions prior to selecting from the provided responses. This section describes the participants, the assignment, and the type of data collected and analyzed for this paper.

2.1 Participants

In the Fall of 2010, students (N=435) enrolled in an introductory computer science course at a major university were assigned a problem set from an online tutoring system as a part of a homework. The course is a first course in Java programming, including topics such as control structures, variables, lists, and simple array algorithms. The course was comprised of a section of students majoring in Computer Science (N=44) as well as sections from two other instructors with non-major students completing a university-wide requirement. Students were given the ability to opt-out of the study; however, they were still required to complete the homework assignment for course credit. Because the study involved random assignment to experimental condition, different questions were provided to students in the same course. In order to mitigate any learning differences across groups, all students were given access to the other tutoring sequences of the study for "extra practice" after completing the required homework.

2.2 The Task

The tutoring system designed for this study presented students with a code snippet and a question about the code on each screen. Students were required to first answer the question by typing an open ended response in a textbox, and then selecting a multiple choice style response from a drop down list. The system automatically provided feedback based upon the answer chosen in the drop down list, and incorrect answers needed to be corrected, by selecting a different response, before the student could continue. The student did not have to re-enter an open ended response in order to progress to the next question. The open ended responses were saved by the system for later analysis by researchers.

The code snippets used for the questions were implementations of 10 array algorithms, and within the tutoring environment there were three questions presented for each code snippet. The first six code snippets were selected to be common algorithms including an implementation of a sum, incrementing all elements in an array, a count of values less than a given, and a max search. The last four code snippets were designed to be more complex and included a sort, and a check for duplicates within the array. Students were randomly assigned to one of four experimental conditions in the tutor, and all students saw multiple choice questions about the same 10 code segments in the same order.

The experimental conditions varied in two ways, which

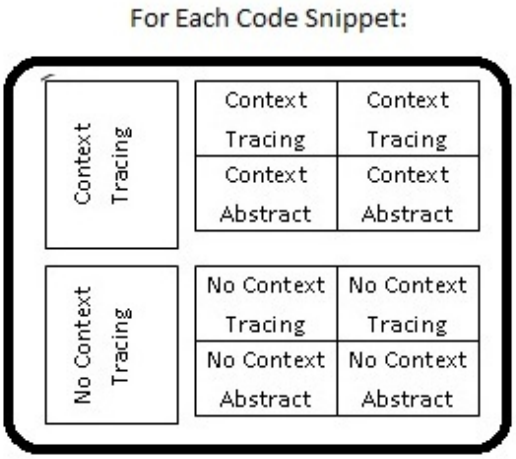

Figure 1: Question Sequence for Each Code Snippet

Code	Description
NR	Non-Responsive Answers categorized as non responsive were single letters, "no answer" or other nonsense words.
MA-C	Matches Answer Correct The student's response matched an answer provided in the multiple choice selections and was correct.
MA-I	Matches Answer Incorrect: The student's response matched an answer provided in the multiple choice selections but was incorrect
NM-C	No Match Correct: The student's response did not match an answer provided in the multiple choice selections, however was an accurate statement.
NM-I	No Match Incorrect: The student's response did not match an answer provided in the multiple choice selection and was incorrect.

Table 1: Classification Codes for Student Responses

are illustrated in Figure 1. Two groups of students saw code with no contextual explanation (i.e., the code simply had a list of numbers and performed some operation), while two groups of students saw the code with an additional sentence offering context for the code snippet (i.e., the list contains the total number of votes cast for each candidate in an election). Two groups of students saw only code tracing questions (i.e., given some values already stored in the list, what is the result of the code), and two groups of students saw one tracing question and two questions asking about algorithmic abstractions (i.e., what is the role of the variable x in the following code) for each code snippet. These variables were crossed yielding four conditions for students, Abstract Questions No Context (AQNC), Abstract Questions with Context (AQC), Tracing Questions No Context (TQNC), and Tracing Questions with Context (TQC). Results of performance across condition are available in [19].

In this paper we analyze the responses students had to questions regarding algorithmic abstraction within the tutoring sequence. Although the students in the experimental conditions with only tracing questions (TQNC, TQC) saw the same code snippets, the open ended responses were the numeric answer produced by the code trace, and not useful for qualitative coding or analysis. We will be focusing on the students in the Abstract Question conditions (AQNC, AQC)(N=113).

2.3 Data Analysis

The student responses were classified based on alignment with any of the possible choices from the multiple choice question, and also on the correctness of the statement made. A response was judged to be correct if it matched the correct option for the multiple choice options provided, or if it made a factual statement relevant to the question asked that did not match one of the incorrect multiple choice options. Table 1 describes the codes employed to classify responses.

Although students contributed an open ended response to every question, in this paper we present the analysis of three particular questions due to their similarity, and the complexity of student responses and analysis. By limiting our question choice, we reduce variance in student responses and in correctness rate due to additional knowledge needed to answer the other questions. The questions we present were asked in regards to code snippets 3 (Sum), 4 (Count) and 5 (Increment). The questions asked about code snippets 3 and 4 ask about the role of a temporary variable used to store the current array element within the for loop in the algorithm. Question 5 asks about the role of the indexing variable (i) which is initialized, and updated as a part of the for loop. By evaluating the performance on question 4 we can determine if the experience of answering question 3 with feedback changed students behavior on the next question. Although question 5 is slightly different, it can be seen as a near-transfer question as the indexing variable is used to access each value in the list, as opposed to storing the value.

2.4 Feedback

Feedback was provided to the students upon selecting from the multiple choice options. The feedback was generated automatically by the system using a platform designed with the Cognitive Tutor Authoring Tools (CTAT) [20]. If the answer was correct, it would turn green and the student would be allowed to proceed to the next question. If the answer was incorrect, it would turn red, a message would appear at the bottom of the screen, and students would need to select the correct answer before continuing in the tutor.

The feedback was constructed to point out any flaws in reasoning that resulted in an incorrect choice. For example, when asked about the purpose of a variable that temporarily represents each element in the list (x = myList[i];) students had the option of choosing "It holds all the values in myList". The correct answer was "It temporarily represents each value in myList" and the feedback presented for the incorrect answer is "Can one variable hold all the values in the list at the same time? What might be a better description for what x does?". See Table 2 for question 3 including choices and feedback.

3. RESULTS

Student responses were analyzed in three ways using a mixed methods approach. First, the responses were qualitatively coded to determine if they matched any of the multiple

Description	Text

Code	public void code(int []myList){
	int x=0;
	for(int i=0; i<myList.length; i++){
	x = x + myList[i];
	}
	return x;
	}

Question	What purpose does the variable x serve in the algorithm implemented above?
Choice	It stores the value 0 (Feedback: That is the starting value of x, does x get changed at any point in the algorithm?)
Choice	It stores the last value in the list (Feedback: The variable x does get changed by the last value in the list, but is that the only value that changes x?)
Choice	To store the sum of all the values in the list (Feedback: This is the final result of the code, is it the purpose of x throughout the execution?)
Choice	It maintains the sum of all the values in the list encountered so far (Feedback: Correct)

Table 2: A Full Problem, The feedback would be displayed after the choice was selected.

Example	Code	Student Response	Explanation
A	NM-I	Represents the number of minutes in one song in the list	The variable represents each element at some point in the algorithm, not just one song.
B	MA-I	It gets added to the variable that represents the sum of the list	While this is a correct statement, it was judged to be a direct translation of the code and not the role of the variable in the algorithm.
C	NM-C	It is the current value to be added	This is a correct statement, however the matching answer references the sum of all values so far.

Table 3: Example Student Responses to Problem 3

choice answers. Then the responses were coded to determine if they were an accurate statement about the code snippet and answering the question asked. Quantitative methods were used to look at the difference in performance across questions and conditions, and to compare the open-ended responses with the multiple choice selection across questions. For this analysis we focus on the three questions described in Section 2.3.

3.1 Results from Qualitative Coding

The results of the coding for the three questions are shown in table 4. Although students received no feedback after typing their open ended response, there was a higher success rate on the first attempt at answering the multiple choice question than the open ended response questions. On question 3, 67% of students wrote a correct open ended response (55% matching the correct option), however 74% of students chose the correct response from the options available. Question 4 was much more consistent with 80% of students providing a correct open ended response (63% matching the correct multiple choice option) and 75% of students choosing the correct option on their first attempt. Question 5 was harder for students with only 34% of students providing a correct open ended response (20% matching the correct multiple choice option) and 37% of students selecting the correct multiple choice option on their first attempt.

As a group, the students who answered question 3 incorrectly showed improvement on question 4. Forty (40) students answered question 3 in an incorrect manner (NM-I, MA-I, NR) in their open ended response, however only 12 of those students answered question 4 incorrectly and of those 7 were non-responsive answers. The pattern is even more interesting because there were two additional questions asked between what we have designated question 3 and question 4 in this paper (one on code base 3 regarding an overall description of the algorithm, and one on code base 4 asking the student to trace the code with given input values).

Table 3.1 contains sample student responses. The No Match Incorrect (NM-I) responses often did not express the algorithms as a process, but instead a singular action. Notice the language in response B. The student indicated that the variable only represented the minutes in one song in the list. These responses echo difficulties students have during code production, where they have trouble understanding or writing algorithmic components with constantly changing state. Similar to the NM-I responses, the Match Incorrect (MA-I) responses did not express either the changing or temporary nature of the variable in the algorithm.

The number of students who incorrectly answered the open-ended question and then correctly answered the multiple choice question could indicate that the constraint of choices acted as feedback to the students about the correct answer, or that some students made appropriate guesses. In Section 4.1 we offer additional distractors appropriate from student responses.

3.2 Statistical Models

Student responses varied across question and experimental condition (No Context or Context). Table 4 shows the number of responses in each category by question. Figure 3.2 shows the number of students whose open ended answer matched a researcher created multiple choice response, whose open ended answer was coded to be accurate, and who answered the multiple choice question correctly on their first attempt. An ANOVA indicates that there is a significant effect of question ($p<.001$) on students' ability to answer correctly, but no significance was attributed to contextual condition ($p=.48$). A linear regression model did indicate significance in matching a multiple choice answer ($p<.05$) in the contextual condition. The regression results indicated that for students in the contextual condition they were less likely to construct an open ended response matching a predetermined distractor. The regression result is an indication

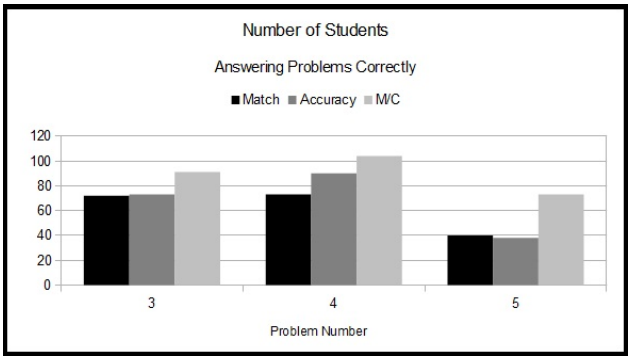

Figure 2: Student Performance by Question

that context may have some impact on student's thought process, and should be studied in more detail.

Student performance increased from question 3 to question 4, indicating some learning had taken place. Student performance decreased for question 5, however it is seen as a near-transfer question, so some performance decrease is expected.

Question	MA-C	MA-I	NM-C	NM-I	NR
No Context 3	37	6	3	7	4
Context 3	23	6	11	13	0
No Context 4	40	2	5	4	6
Context 4	31	0	14	8	3
No Context 5	15	10	1	27	7
Context 5	8	9	15	20	4

Table 4: Student Performance by Question

Question 3 Student performance on the first attempt of the multiple choice response of question 3 was only significantly affected by whether their response matched an answer available in the multiple choice selections ($p<.05$). The condition that the student was in, and the accuracy of the statement made by the student was not significant with respect to answering the multiple choice selection.

Question 4 Student performance on the first attempt multiple choice response of question 4 was significantly impacted by their correct choice response in question 3 ($p<.001$) and whether their open ended response for question 4 was categorized as accurate ($p<.001$). Eighty-six (86) percent of students who answered the multiple choice option incorrectly for question 3, answered the multiple choice option correctly for question 4. While it may be argued that the students just remembered the last option (even with the questions between) and re-chose it, there is evidence that it was more than simple recognition. Of the students who answered question 3 incorrectly, and then question 4 correctly, 74% of them also wrote a correct open-ended response as well.

Question 5 Student performance on the first attempt multiple choice response of question 5 was significantly impacted by their correct choice response in question 3 ($p<.01$), their accuracy in the open ended part of question 4 ($p<.05$) and their accuracy in the open ended part of question 5 ($p<.01$).

4. CONCLUSIONS

The significant performance improvement from the first to second question indicates that code comprehension questions can be used as learning activities when accompanied by feedback. Just as a teacher would use a worked example in class, and use the class to answer questions regarding the example, code comprehension activities can be used as a part of problem sets to highlight features of code. Future work is planned to evaluate the impact of code comprehension learning activities on code production with introductory students.

4.1 Using Open-Ended and Multiple Choice

Together, the first and third authors of this paper have over 30 years of teaching experience in both high school and college settings. Although the incorrect responses were designed to be effective distracters, analysis of the open ended responses provided data indicating that students had alternative models of the code. As online and automated assessment becomes increasingly popular, the use of multiple choice items for their ease of grading will be common. We hope that this work highlights the additional data collection that can be done using open ended responses, not to be used in scoring the students, but instead to offer assessment designers a way to evaluate the distracters used in multiple choice style questions.

Within the three questions analyzed in this paper, the most common incorrect answers that did not match a response shared language referring to "an element" or "the value". This would indicate the need for a distractor focused on a singular state held by variables that are updated as the loop continues through the algorithm. If these questions are reused in a future study we will include a response in the multiple choice answers addressing this misconception.

4.2 Recommendations for the Use of Worked Examples

Computer programming requires reasoning on multiple levels of abstraction during problem solving. In order to comprehend code, novices must understand (parse) individual commands, and attempt to construct a larger understanding of the algorithm as a whole[21, 18]. Novices often can understand the individual parts, but struggle with formulating a coherent complex representation of all of the parts of the algorithm[3].

The responses discussed in this paper highlight a particular misconception that students struggle with, the "in motion" properties of algorithms. Unlike the solving of algebraic expressions in mathematics classes, the addition of looping constructs means that a single variable, in the same line of the worked example, can represent multiple values. Algorithmic animations do a good job of emphasizing the change, especially if they include simultaneous code highlighting.

For instructors and textbook authors who present worked examples in code, emphasis on the change that occurs to variables over time within the algorithm is recommended. The authors plan to explore this misconception and appropriate instructional methods in future studies.

5. ACKNOWLEDGMENTS

The authors would like to thank Matt Jadud for helping to facilitate access to his students and computers at Allegheny

College. This work was supported through the Program for Interdisciplinary Education Research (PIER) at Carnegie Mellon University, funded through Grant R305B040063 to Carnegie Mellon University, from the Institute of Education Sciences, US Department of Education. The opinions expressed are those of the authors and do not represent the views of the Institute or the US Department of Education.

6. REFERENCES

[1] Carsten Schulte, Teresa Busjahn, Tony Clear, James Paterson, and Ahmad Taherkhani, "An introduction to program comprehension for computer science educators", in *ITiCSE '10: Proceedings of the fifteenth annual conference on Innovation and technology in computer science education*, New York, NY, USA, 2010, pp. 108–112, ACM.

[2] Mike Lopez, Jacqueline Whalley, Phil Robbins, and Raymond Lister, "Relationships between reading, tracing and writing skills in introductory programming", in *ICER '08: Proceeding of the Fourth international Workshop on Computing Education Research*, New York, NY, USA, 2008, pp. 101–112, ACM.

[3] Raymond Lister, "The neglected middle novice programmer: Reading and writing without abstracting", *In Proceedings of the 20th Annual Conference of the National Advisory Committee on Computing Qualifications*, 2007.

[4] John Sweller and Graham Cooper, "The use of worked examples as a substitute for problem solving in learning algebra", *Cognition and Instruction*, vol. 2, pp. 59–89, 1985.

[5] Beth Simon, Michael Kohanfars, Jeff Lee, Karen Tamayo, and Quintin Cutts, "Experience report: peer instruction in introductory computing", in *Proceedings of the 41st ACM technical symposium on Computer science education*, New York, NY, USA, 2010, SIGCSE '10, pp. 341–345, ACM.

[6] Raymond Lister, Tony Clear, Simon, Dennis J. Bouvier, Paul Carter, Anna Eckerdal, Jana Jacková, Mike Lopez, Robert McCartney, Phil Robbins, Otto Seppälä, and Errol Thompson, "Naturally occurring data as research instrument: analyzing examination responses to study the novice programmer", *SIGCSE Bull.*, vol. 41, pp. 156–173, January 2010.

[7] Raymond Lister, Elizabeth S. Adams, Sue Fitzgerald, William Fone, John Hamer, Morten Lindholm, Robert McCartney, Jan Erik Moström, Kate Sanders, Otto Seppälä, Beth Simon, and Lynda Thomas, "A multi-national study of reading and tracing skills in novice programmers", *SIGCSE Bull.*, vol. 36, pp. 119–150, June 2004.

[8] Laurie Murphy, Renee McCauley, and Sue Fitzgerald, "'explain in plain english' questions: Implications for teaching", *Proceedings of the fourty-third ACM Special Interest Group on Computer Science Education*, 2012.

[9] A. Philpott, P. Robbins, and J. Whalley, "Assessing the steps on the way to relational thinking", *Proceedings of the 20th annual conference of the National Advisory Committee on Computing Qualifications*, p. 286, 2007.

[10] T. Clear, J. Whalley, P. Robbins, A. Philpott, A. Eckerdal, M. Laakso, and R. Lister, "Report on the final bracelet workshop", *Journal of Applied Computing and Information Technology*, vol. 15 (1), 2011.

[11] National Research Council, *How People Learn: Brain, Mind, Experience and School*, National Academy Press, 2000.

[12] Steve Ritter, John Anderson, and Ken Koedinger, "Cognitive tutor: Applied research in mathematics education", *Psychonomic Bulletin & Review*, vol. 14(2), pp. 249–255, 2007.

[13] J. Anderson and B. Reiser, "The lisp tutor: it approaches the effectiveness of a human tutor", *Lecture*, vol. 174, 1985.

[14] C. Areias and A. Mendes, "A tool to help students to develop programming skills", *Proceedings of the 2007 International Conference on Computer Systems and Technologies*, 2007.

[15] Nick Parlante, "Codingbat: Practice java and python problems".

[16] Elliot Soloway, Beverly Woolf, Eric Rubin, and Paul Barth, "Meno-ii: an intelligent tutoring system for novice programmers", *Proceedings of IJCAI'81, Proceedings of the 7th international joint conference on artificial intelligence*, 1981.

[17] Anne Venables, Grace Tan, and Raymond Lister, "A closer look at tracing, explaining and code writing skills in the novice programmer", in *ICER '09: Proceedings of the fifth international workshop on Computing education research workshop*, New York, NY, USA, 2009, pp. 117–128, ACM.

[18] Jacqueline L. Whalley, Raymond Lister, Errol Thompson, Tony Clear, Phil Robbins, P. K. Ajith Kumar, and Christine Prasad, "An australasian study of reading and comprehension skills in novice programmers, using the bloom and solo taxonomies", in *Proceedings of the 8th Austalian conference on Computing education - Volume 52*, Darlinghurst, Australia, Australia, 2006, ACE '06, pp. 243–252, Australian Computer Society, Inc.

[19] Leigh Ann Sudol-DeLyser and Jonathan Steinhart, "Factors impacting novice code comprehension in a tutor for introductory computer science", *In Proceedings of the 2nd Annual Conference in Educational Data Mining.*

[20] Vincent Aleven, Bruce McLaren, Johnathan Sewall, and Ken Koedinger, "A new paradigm for intelligent tutoring systems: Example-tracing tutors", *International Journal of Artificial Intelligence in Education*, vol. 19(2), pp. 105–154, 2009.

[21] Carsten Schulte, "Block model: an educational model of program comprehension as a tool for a scholarly approach to teaching", *Proceedings of the Fourth Annual Workshop on Computer Science Education*, 2008.